



Pager Rotation Duties

Clint Scott

CSD380 DevOps

Module 7.2 Assignment – Pager Rotation Duties

November 30, 2025

You Build It, You Run It

Shared Responsibility

According to The DevOps Handbook (Chapter 16), integrating development into on-call rotations is transformative.

When developers share pager duties with Ops, they gain immediate feedback on code quality and system reliability. This "feedback loop" incentivizes writing cleaner, more resilient code and reducing technical debt, as the people writing the bugs are the ones woken up by them.



Designing the Schedule

Rotation Frequency

Weekly: Best for teams of 4+ engineers.
Provides a full week of focus followed by weeks of off-time.

Daily/Split: Better for small teams (2-3 people) to prevent exhaustion from week-long shifts.



Follow-the-Sun

For global organizations, this model is the gold standard. Teams in different time zones cover their own daylight hours.

This eliminates the “graveyard shift,” significantly reducing burnout and improving alert response times.



Tiered Response Structure



Primary On-Call

The first line of defense. Acknowledges alerts within minutes. Focuses purely on triage and stabilization during their shift.



Secondary On-Call

The backup. If the Primary misses an alert or gets overwhelmed, the page escalates here. Crucial for redundancy.



Shadow Rotation

A training role for new engineers. They receive alerts but don't respond, allowing them to learn the process without pressure.

Combating Alert Fatigue

Human-Centric Policies

Compensation

Offer extra pay or "Time Off In Lieu" (TOIL) for on-call hours.



Actionable Alerts

If a human can't do anything about it, it shouldn't be a page. Delete "information-only" alerts.



Minimum Team Size

Avoid 2-person rotations. A minimum of 3-4 engineers is needed to prevent rapid burnout.



Easy Swaps

Life happens. Tools should allow seamless shift trading for emergencies or personal time.



The Cultural Foundation

“You can’t fire your way to reliability.
We need to create a culture where
people feel safe to admit mistakes.”

– Site Reliability Engineering Principles

Implementing **Blameless Post-Mortems** shifts the focus from "Who caused this?" to "How did the system allow this to happen?" This psychological safety is essential for effective incident response.



The Incident Lifecycle



Detection

Automated monitoring triggers an actionable alert to the Primary On-Call.



Triage

Responder acknowledges and assesses severity. Communication channel established.



Remediation

Fix or workaround applied to restore service. Escalation if needed.



Review

Blameless post-mortem analysis to prevent recurrence and update runbooks.

Essential Tooling



Monitoring

Prometheus, Datadog, New Relic



Paging Service

PagerDuty, OpsGenie, VictorOps



Automation

Ansible, Runbooks, ChatOps

Summary



Shared Ownership

Devs on-call improves code quality.



Health First

Manage fatigue with fair scheduling and tools.



Safety

Blameless culture enables learning.

References

- Atlassian. (n.d.). *A better approach to on-call scheduling*. Retrieved from <https://www.atlassian.com/incident-management/on-call/on-call-schedules>
- Google. (2016). Being on-call. In *Site Reliability Engineering: How Google runs production systems*. O'Reilly Media. Retrieved from <https://sre.google/sre-book/being-on-call/>
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press.
- PagerDuty. (n.d.). *PagerDuty Best Practices for On Call Teams Ops Guide*. Retrieved from <https://goingoncall.pagerduty.com/>